Operating Systems Lab 8 - Memory Management in xv6
Student: Tejeshwar Bhagat | Roll number: 2401MC23
Base: MIT xv6-public commit eeb7b415dbcb12cc362d0783e41c3d1f44066b17

Q1 - Virtual to physical address translation

xv6 uses two-level paging. va2pa walks the calling process page table, checks that th
is present and user-accessible, then combines the frame address with the 12-bit pag
It returns -1 for absent pages, supervisor-only pages, and addresses in the kernel ra
The kernel prints the page-directory index, page-table index, offset, and P/W/U flags.

vatest requests translations for a global, local, and malloc-backed variable. It checks
unmapped address 0x7fffffff and forks. Parent and child keep the same virtual addres
the global variable; copyuvm creates separate physical frames. Expected: equal VAs
different PAs.

Run: vatest

Q2 - Physical memory statistics

kalloc.c tracks the number of pages on the physical free list. kfree increments the count and kalloc decrements it while kmem.lock is held during normal multiprocessor operation. The bootstrap phase keeps xv6's lock-free initialization. ktotalpages reports the frames managed by this allocator, excluding reserved memory and the kernel image.

The shared meminfo structure contains total_pages, free_pages, user_pages, and page_table_pages. user_pages is sz / PGSIZE. page_table_pages counts the process page directory and each present page-table page, including kernel mappings.

memtest prints statistics before allocation, after it grows and touches two heap pages, inside the forked child, and after the child is reaped. Expected: free pages decrease after allocation and fork, then return to the post-allocation level after wait reclaims the child. The test prints exact values during a QEMU run; no measured values are claimed here.

Runtime values to transcribe from memtest:

Before allocation: free_pages = (capture from run)
After touching 2 pages: free_pages = (capture from run)
Child after fork: free_pages = (capture from run)
After wait: free_pages = (capture from run)

A free-list counter must cover both allocator initialization ranges and remain consistent when child pages are released. Updating it at list insertion and removal keeps the count aligned with the allocator's actual state.

Run: memtest

Q3 - Dumping the process page table

vmprint walks the process page directory. For each present user PDE it prints the page-table physical address and flags. It then prints each present PTE with its virtual page, physical frame, and P/W/U flags. Kernel mappings at or above KERNBASE are omitted from the detailed dump and included in a one-line count.

exec records the stack guard address while building a process image. vmprinttest prints the expected guard VA so it can be found in the dump: the PTE is present (P=1) and inaccessible to user mode (U=0). At startup, user-accessible pages should equal sz / PGSIZE minus the guard page.

vmprinttest captures the initial page table, reserves eight pages, touches one byte in each, then prints again. It shrinks the heap by one page and prints a final dump. The touched pages
should appear after growth and the final page should disappear after shrink. Empty page-table
pages remain allocated until the address space is freed, matching xv6 deallocuvm behavior.

Because Q4 changes sbrk to lazy allocation, this test touches each reserved page before checking the expanded dump.

Run: vmprinttest

Q4 - Lazy heap allocation

Positive sbrk extends proc->sz without allocating frames. Each process tracks heapbase and stackguard. A page fault is resolved only for a missing page in the heap, below sz and above the guard. The kernel allocates a zero-filled frame and maps it writable and user-accessible.
Invalid user addresses and allocation failures terminate the faulting process.

copyuvm skips absent PTEs so fork works with untouched heap holes. deallocuvm skips absent leaves and frees only allocated frames. Shrinking below the original heap boundary is rejected to protect the stack and program image. Syscall argument validation materializes legal lazy pages before kernel I/O accesses user buffers, covering untouched read/write buffers.

lazytest reserves 32 MB, prints free-page statistics before and after reservation, touches four distant pages, forks across remaining holes, touches another missing page in the child, passes an untouched page to write, and has a child access an address beyond sz. Expected: no frame use for reservation alone; frames allocated on touch; fork survives sparse pages; and only the invalid child is terminated.

Verification record:
- Changed C files: 32-bit GCC syntax check passed.
- Full xv6 build / QEMU run: not run; make, i686-elf tools, and QEMU are unavailable here.
- Measured memory values and terminal screenshots: pending a real QEMU run.

Run: lazytest